مِرآة

استوديو الصور المتقدم | Mirror Image Studio

محرر صور احترافي يعمل عبر الويب — واجهة Frontend وواجهة Python Backend

مشروع معالجة الصور الرقمية (Digital Image Processing)

المقرر: معالجة الصور الرقمية · قسم تقنية المعلومات (IT4)

إعداد: فريق المشروع (١–٣ أعضاء)

عرض تقديمي (Demo) · تحليل تقني شامل · عرض عملي

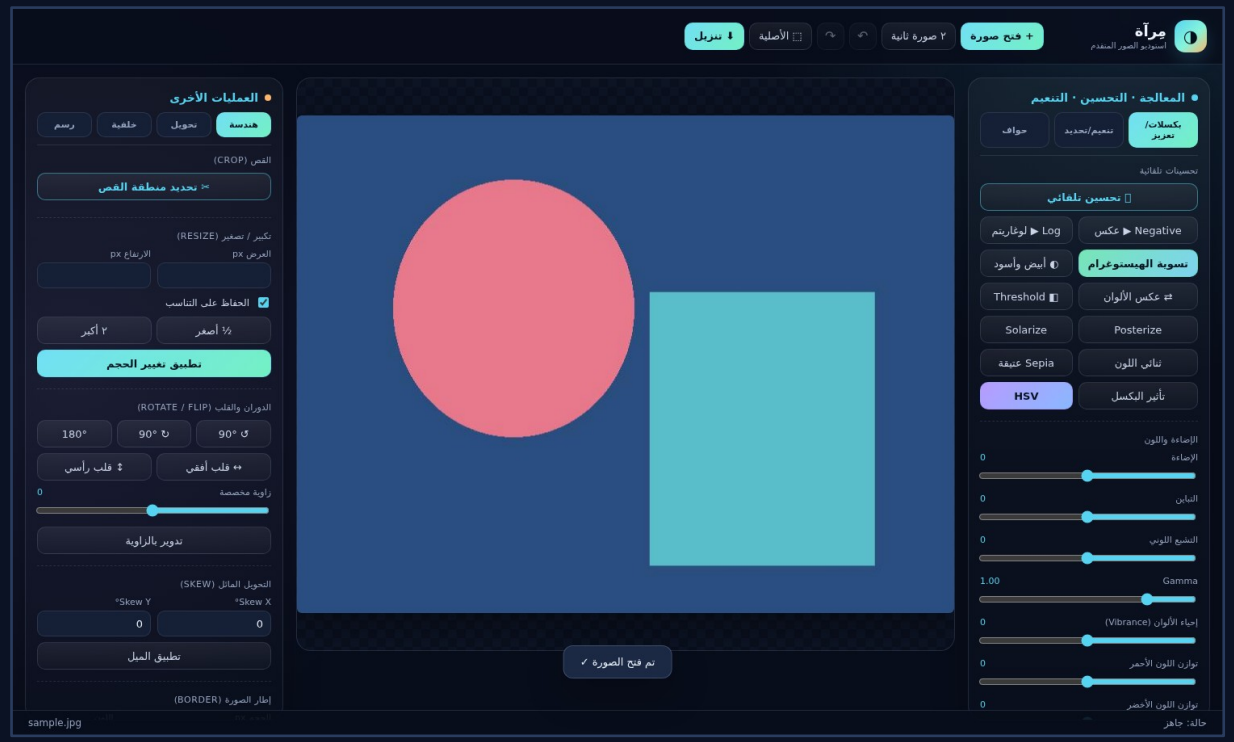
فكرة المشروع

ما هو؟ · ما الهدف؟ · ماذا يفعل؟

المفهوم

- **أداة ويب** لتحرير الصور بضغطة زر
- **المبدأ:** أدخل صورة ← اختر عملية ← احصل على النتيجة
- **تعليمي وعملي:** يطبّق خوارزميات معالجة الصور الرقمية
- **واجهة عربية** كاملة وسهلة الاستخدام
- **لا يحتاج** تسجيل دخول أو قاعدة بيانات

لقطة حقيقية من المشروع (بعد رفع صورة) ◂



المشكلة والحل

لماذا هذا المشروع مفيد؟

المشكلة

- برامج التحرير الاحترافية معقّدة ومكلفة
- تعلمها يستغرق شهوراً
- المفاهيم النظرية للمعالجة صعبة التخيّل بلا تجربة



الحل

- محرر ويب مبسّط بأشهر العمليات
- كل عملية = زر واحدبضغطة واحدة
- أمثلة مباشرة لأفكار الخوارزميات
- خلفيات منتجات + إزالة خلفية جاهزة للتسويق

وظائف المشروع

أهم ما يستطيع المستخدم فعله (موجود فعلاً في الكود)

التحويل والصيغ

- PNG / JPG / WEBP / BMP / GIF
- ضبط جودة الحفظ

العمليات الهندسية

- قص Crop
- تكبير/تصغير Resize
- تدوير Rotate
- قلب Flip
- إطار Border

تحسينات البكسل

- إضاءة / تباين / تشبع
- Gamma · HSV
- أبيض وأسود · عكس
- تسوية هيستوغرام · Log · Negative

فلاتر التنعيم والتحديد

- Mean · Gaussian (خطي)
- Median (غير خطي)
- Custom نواة مخصصة
- Sharpen · Unsharp · Emboss

كشف الحواف

- Laplacian · Laplacian Edge
- Difference
- Sobel (X/Y/مقدار)
- Canny (من الصفر)

إبداعية ومتقدمة

- إزالة الخلفية
- دمج صورتين
- مكتبة خلفيات منتجات
- رسم + نص + مؤثرات

المعمارية Architecture

بنية Client–Server حقيقية من المشروع



ملاحظات

- **لا قاعدة بيانات:** كل شيء في الذاكرة
- **لا مصادقة:** مقبول محلياً
- **مكتبات:** Pillow + NumPy + http.server
- **الواجهة** HTML/CSS/JS خام

التقنيات المستخدمة

كل تقنية مذكورة موجودة فعلاً في المشروع

أدوات البناء

HTML5
بناء هيكل الواجهة والأزرار

CSS3
التصميم والتبويبات والمظهر

JavaScript
منطق الواجهة والتفاعل وإرسال الطلبات

Python
قاعدة الخادم وكل معالجة الصور

http.server
خادم HTTP (بلا إطار عمل) قياسي

Pillow / PIL
فتح وتحرير وحفظ الصور

NumPy
العمليات الحسابية والالتفاف

Canvas API
عرض الصورة والرسم داخل المتصفح

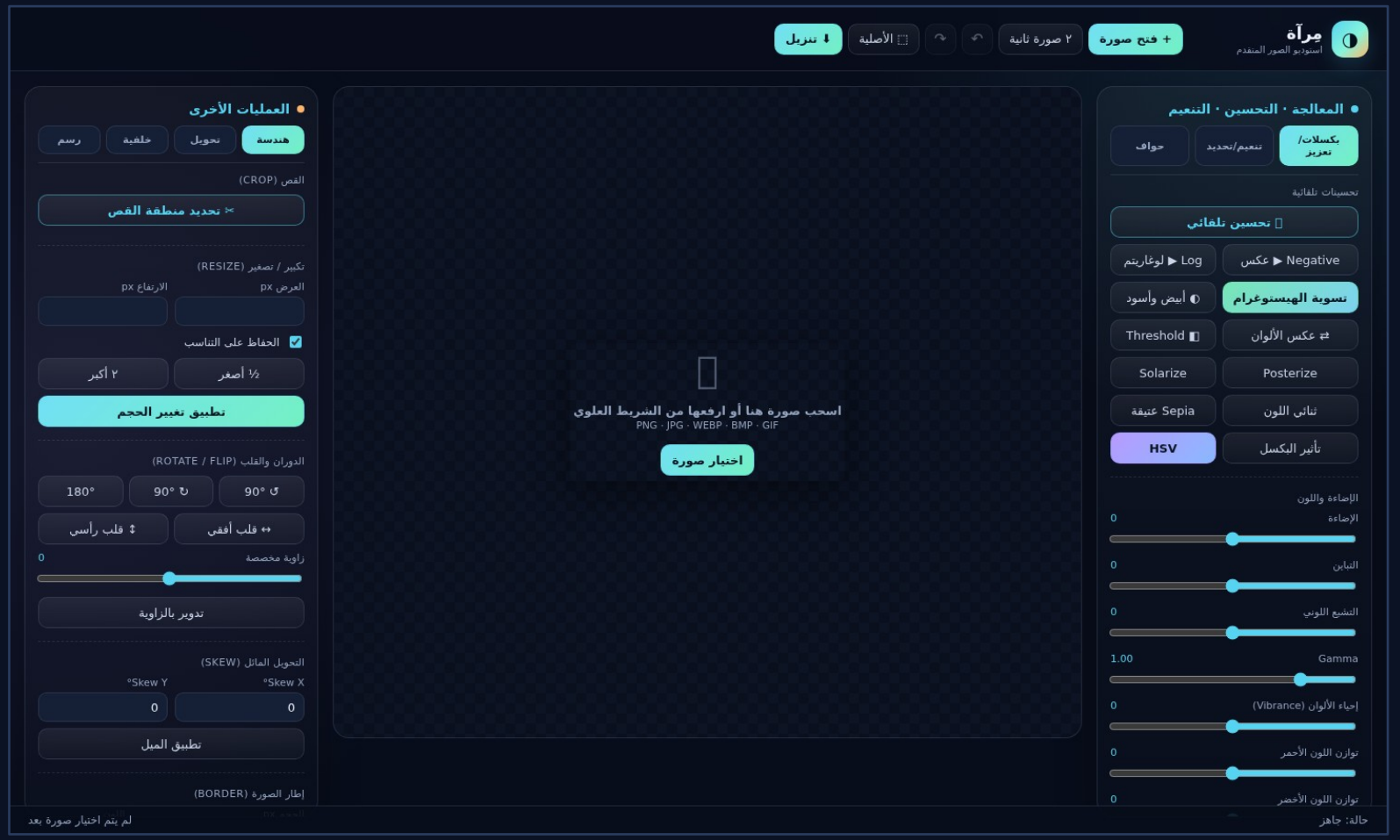
fetch / JSON
التواصل بين الواجهة والخادم

ملاحظة: توجد مكتبة rembg مذكورة كخيار ذكاء اصطناعي اختياري، لكنها غير مفعّلة وغير مثبّتة — يُستخدم البديل الخفيف تلقائياً.

UI واجهة المستخدم

لقطات حقيقية من المشروع الجاري

الشاشة مقسّمة إلى: لوحة المعالجة (يسار) · منطقة الصورة (وسط) · لوحة العمليات (يمين)



رحلة المستخدم User Flow

كيف يستخدم المستخدم المشروع خطوة بخطوة



ماذا يحدث في الحالات غير الطبيعية؟

- **ننفي أولاً:** إن ضغط زر عمليات بلا صورة يظهر «افتح صورة أولاً» ولا يتلف شيء
- **ملف غير صالح:** الخادم يرفضه ويعيد «لم تصل صورة صالحة»
- **حجم مبالغ:** تُرفض الطلبات فوق 64 ميجابايت
- **أثناء المعالجة:** تظهر «جارٍ المعالجة...» وتُقفل أزرار التنفيذ مؤقتاً

Frontend الواجهة الأمامية

ملف index.html — كل المنطق داخل وسم <script>

- **الملف الوحيد:** index.html
- **HTML/CSS** بناء الأزرار والتبويبات والتصميم
- **JavaScript** منطق التفاعل كاملاً
- **Canvas** عرض الصورة والرسم المباشر
- **FileReader** قراءة الصورة المرفوعة
- **fetch()** إرسال الطلبات للخادم
- **state** كائن يدير الصورة + التراجع/الإعادة
- **history/future** مكدسّا التراجع والإعادة (بحد 60)

مقتطف حقيقي من الكود — إرسال طلب العملية

```
async function processImage(op, extra = {}){
  if(!hasImage()){ toast("افتح صورة أولاً", true); return; }
  setBusy(true);
  try {
    const body = { op, image: state.current, ...extra };
    const res = await fetch("/api/process", {
      method:"POST",
      headers:{"Content-Type":"application/json"},
      body: JSON.stringify(body)
    });
    const out = await res.json();
    if(!out.ok) throw new Error(out.error);
    await commitData(out.data, "تم تطبيق: " + op);
  } catch(e){ toast(e.message||"حدث خطأ", true); }
  finally { setBusy(false); }
}
```

الواجهة الخلفية Python Backend

ملف app.py — الخادم + الخوارزميات

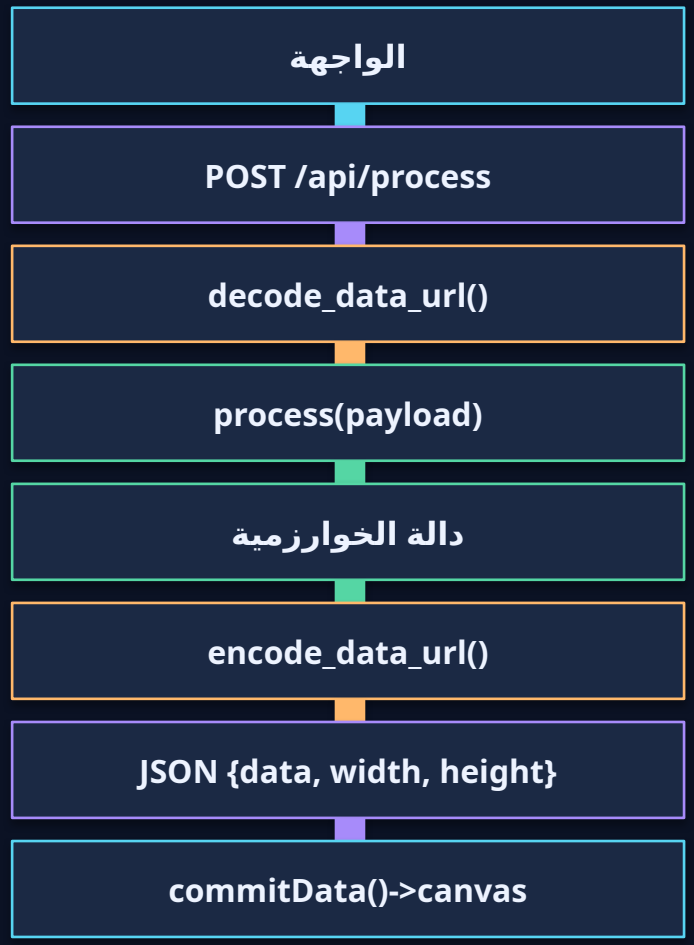
- **framework:** http.server القياسي (بلا Flask)
- **الدخول:** main() → ThreadingHTTPServer
- **الكلاس:** ImageEditorHandler
- **Routes:** / · /api/health · /api/backgrounds
- **API:** /api/process (POST) + /backgrounds/<name> (GET)
- **process(payload):** الموجّه الرئيسي للعمليات
- **دوال معالجة:** apply_canny, blend_images, remove_background…
- **encode/decode:** تحويل الصور base64

مقتطف حقيقي — استقبال الطلب ومعالجته

```
                                    try:
    length = int(self.headers.get("Content-Length","0"))
    if length <= 0 or length > MAX_BODY:
        self.send_json({"ok":False,"error":"حجم الطلب كبير جداً"},413)
        return
    payload = json.loads(self.rfile.read(length).decode("utf-8"))
    result, output_format = process(payload)
    data = encode_data_url(result, output_format,
payload.get("quality",92))
    self.send_json({"ok":True,"data":data,
                    "width":result.width,"height":result.height})
except ValueError as exc:
    self.send_json({"ok":False,"error":str(exc)},400)
except Exception as exc:
    self.send_json({"ok":False,"error":str(exc)},500)
```

التواصل Frontend ↔ Backend

رحلة الطلب الكاملة + انتقال الصورة



state.current (dataURL)

JSON: {image, op, ...}

dataURL → Image (RGBA)

توجيه العملية بالاسم op

apply_* / blend / remove

Image → dataURL

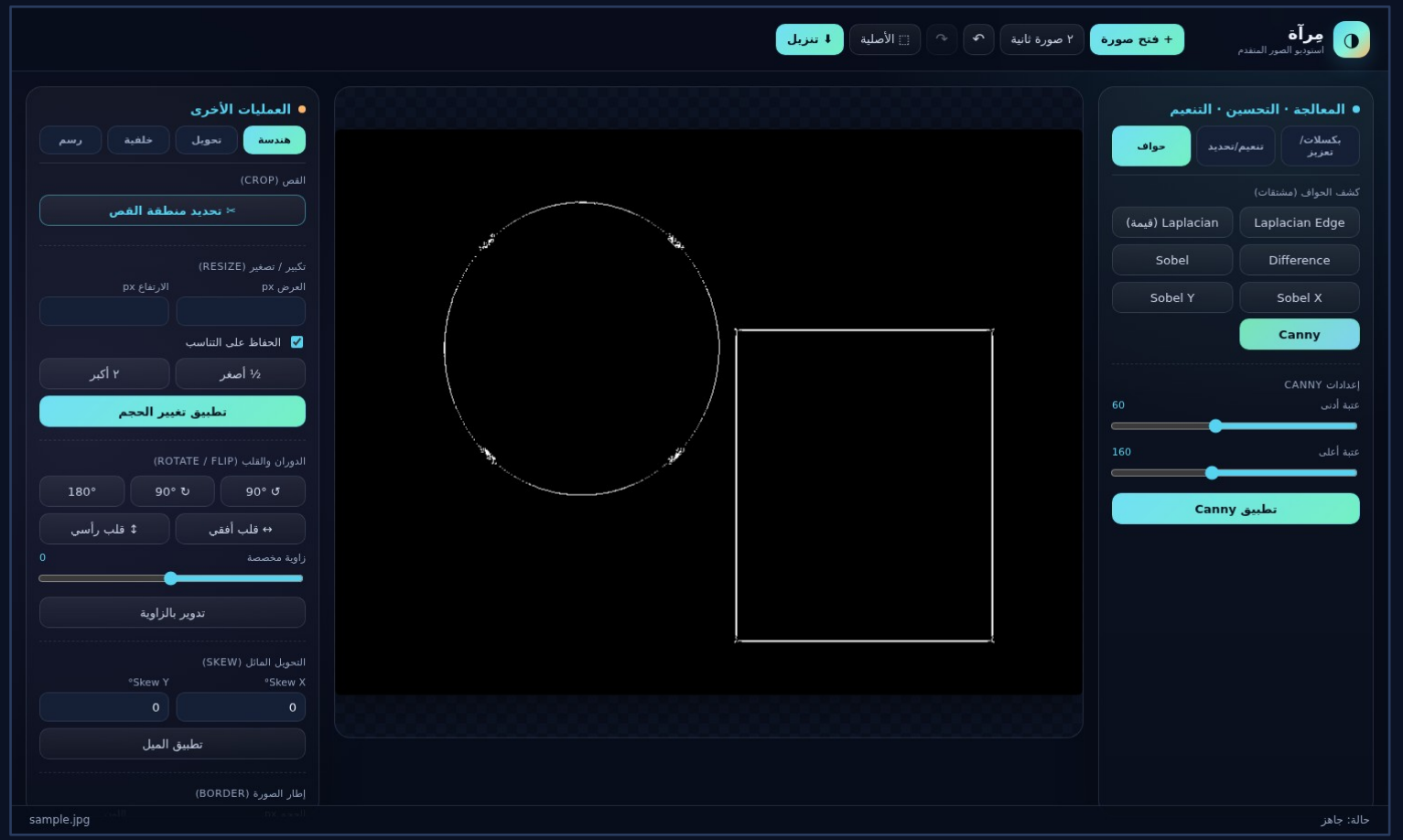
يرجع إلى الواجهة

عرض النتيجة

معالجة الصور Image Processing

أمثلة حقيقية — Input → Processing → Output

على صورة مرفوعة (كشف الحواف) Canny لقطة حقيقية: تطبيق خوارزمية



أهم أجزاء الكود

4 أجزاء تفسّر طريقة عمل المشروع

process(payload)

يقرأ — ويستدعي الخوارزمية op الموجّه الرئيسي

apply_canny()

كشف حواف من الصفر: غاوسي→Sobel→NMS→Hysteresis

remove_background_simple()

إزالة الخلفية عبر Flood-Fill لوني

state + undo/redo

إدارة الحالة والتراجع في الواجهة

apply_canny() (من الصفر) — مقتطف حقيقي

```
                    gray = grayscale_array(image)
smooth = GaussianBlur(gray)      # 1) تنعيم
sx, sy = sobel(smooth)           # 2) تدرج
mag = sqrt(sx**2 + sy**2)
ang = degrees(arctan2(sy,sx))%180
# 3) Non-Maximum Suppression
for y in range(1,h-1):
  for x in range(1,w-1):
    if mag[y,x] >= neighbors:
        nms[y,x]=mag[y,x]
# 4) Hysteresis (ربط الحواف)
# يتتبع الحواف الضعيفة
# المتصلة بحافة قوية عبر deque
```

النتائج والتحديات

ما الذي تحقق؟ وما حدود المشروع وما النواقص؟

ما تم إنجازه فعلاً

- **خادم HTTP** يعمل ويعالج الطلبات
- **كل عمليات المحرر** تعمل (تحققنا بنفسنا)
- **خوارزميات المحاضرات** الخمس مطبّقة
- **يكتشف** الحواف فعلاً (أثبتناه بلقطة) **Canny**
- **واجهة عربية** تعمل بضغطة زر

الحدود والنواقص

- **لا قاعدة بيانات** ولا حفظ دائم
- **لا مصادقة** ولا صلاحيات
- **إزالة الخلفية** الخفيفة تنجح مع الخلفيات المتجانسة فقط
- على الصور الكبيرة Canny **بطء في** الحلقات للـ
- خطأ في) **صفحة أبعاد الصورة** لا تُعرض (updateMeta)
- **دالة np_from** معرّفة لكن غير مستخدمة

الخاتمة والتطوير المستقبلي

نقاط القوة · ما يمكن إضافته

نقاط القوة

- **بساطة التشغيل** (ملفان فقط)
- **واجهة عربية** سهلة
- **تغطية شاملة** لمنهج معالجة الصور
- **فصل نظيف** بين الاتصال والمعالجة
- **ميزات حقيقية** مفيدة للتسويق

تطويرات مستقبلية منطقية

- **إصلاح** خطأ عرض الأبعاد
- **تفكيك app.py** إلى وحدات منفصلة
- **تحويل process()** قابل للتوسع OPS إلى قاموس
- **تسريع** الالتفاف بمكتبة متجهة
- **إضافة** طبقات Layers ومقارنة قبل/بعد
- **دعم** AI حقيقي لإزالة الخلفية

شكراً لحسن الاستماع

أسئلة وتعليقات؟

مِرآة — استوديو الصور المتقدم

لِتجربة المشروع

```
python app.py
open the browser at http://localhost:8000
run: pip install pillow numpy
```